带 Token 鉴权的迷你 REST API

用 socket 写一个能登录、能留言的 HTTP 服务

网络编程课程设计

C11　POSIX socket　2026

今天讲四件事

1. 服务要完成什么
2. 一个连接怎么接进来，一条请求怎么切出来
3. token 和留言放在哪，为什么不能放在普通变量里
4. 用三条 curl 把 Week A 到 Week C 跑一遍

现场顺序也是这四步：先看进程模型，再看报文边界，最后开两个终端演示。

课设要交的最小功能

- `POST /api/login`：用户名和密码写死三个账号，成功就返回一枚 token
- `GET /api/messages`：请求头里带 `Authorization: Bearer <token>`，返回留言列表
- `POST /api/messages`：同样要 token，把一条留言存起来，返回 201
- 没带 token，或者 token 不对，返回 401

传输层自己做：`socket`、`bind`、`listen`、`accept`，一个连接一个进程。
不引入 HTTP 库，也不做网页。

接口和账号

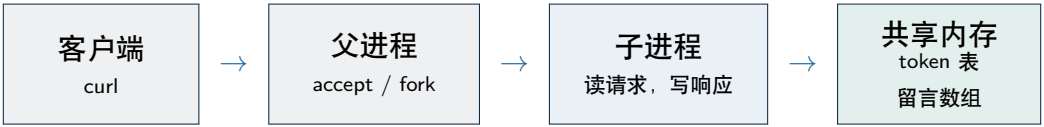
方法	路径	作用
GET	`/health`	打通读写
GET	`/api/ping`	心跳，加分
POST	`/api/login`	校验账号，发 token
GET	`/api/messages`	读留言，要 token
POST	`/api/messages`	写留言，201

三个写死的账号

alice / alice123
bob / bob123
carol / carol123

留言板是全局的。token 用来证明你是谁，作者名从 token 查，不从 body 里取。

进程和数据放在哪



父进程只负责接入和回收子进程。
token 和留言若放在普通全局变量里，`fork` 之后每个子进程改的是自己的副本。

代码地图

文件	负责
server.c	监听、fork、路由
http.c	读满一条请求，写响应
json.c	取一层字符串字段
auth.c	账号和 token 表
store.c	留言循环数组
state.c	共享内存和锁

编译和演示

```
make
./bin/restapi 8080
bash scripts/demo.sh
```

演示脚本先跑课设要求的三条 curl，再检查 401、伪造作者和两个客户端同时写。

一次请求的八步

1. 父进程 `accept` 得到连接
2. `fork`：子进程关监听套接字，父进程关连接
3. `read`，直到出现空行
4. 按 `Content-Length` 把 body 读满
5. 看第一行的方法和路径
6. 需要保护的接口去查 token 表
7. 按固定模板写回 JSON
8. 关掉连接，子进程结束

一个连接只处理一个请求。处理完子进程就退出，所以父进程必须负责回收。

一个连接一个进程

```
pid = fork();
if (pid == 0) {
    close(listen_fd);
    set_timeouts(conn);
    handle_client(conn, state);
    close(conn);
    _exit(0);
}
close(conn);
```

主干在 server.c。fork 前后会先挡住信号。

- 父进程堵在 accept，轮不到 waitpid
- SIGCHLD 里回收，避免僵尸进程
- SA_RESTART 让回收不打断 accept
- 子进程用 _exit，不重复做清理
- 忽略 SIGPIPE，对端断开不会把子进程打死

TCP 没有消息边界



先读到 \r\n\r\n，再读 Content-Length 个字节。两次 read 可能把头部从中间切开，也可能把头和 body 粘在同一次返回里。

```
sep = strstr(buf, "\r\n\r\n");
need = header_end + 4 + content_length;
while (len < need) {
    n = read(fd, buf + len, cap - len);
    len += (size_t)n;
}
```

上面是边界判断的形状，完整循环在 http.c。不支持分块传输。两份 Content-Length 直接 400。

响应模板和状态码

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: N
Connection: close

{"ok":true}
```

课设模板是前三行加空行加 JSON。

多写 `Connection: close`，因为 HTTP/1.1 默认还会再等下一个响应。

状态码	何时
200	登录、读取、health、ping
201	留言已写入
400	JSON 或请求不完整
401	没 token、token 错、密码错
404	没有这个路径
405	路径对，方法不对
408	读请求超时
413	头部或 body 超长

token 怎么签发、怎么核对

```
seed = (unsigned)time(NULL)
     ^ (unsigned)getpid()
     ^ (unsigned)clock() ^ salt;
srand(seed);
sprintf(out, "tk_%08x%08x",
        (unsigned)rand() ^ (unsigned)getpid(),
        (unsigned)rand() ^ salt);
```

1. 账号密码和表里的三项做比较
2. sprintf 出 16 位十六进制串
3. 写入共享内存里的 token 表
4. 之后用 Bearer 后面的串查表

fork 会复制 rand 的状态。不按 pid 重新播种的话，两个子进程可能发出同一枚 token。

密码错误、用户不存在，都返回同一句「用户名或密码错误」。

为什么必须用共享内存

只用全局变量会怎样

登录写在子进程自己的数组里。下一个连接是另一个进程，查的是另一份表，刚发的 token 一定无效。留言同理。

崩溃时锁不能永远占着

子进程如果死在持锁期间，普通互斥锁会把整张表锁死。这里用了可恢复的进程间锁，下一个加锁的人把锁收回来。

```
st = mmap(NULL, sizeof(*st),
    PROT_READ | PROT_WRITE,
    MAP_SHARED | MAP_ANONYMOUS,
    -1, 0);
pthread_mutexattr_setpshared(
    &attr, PTHREAD_PROCESS_SHARED);
pthread_mutexattr_setrobust(
    &attr, PTHREAD_MUTEX_ROBUST);
```

映射在 `fork` 之前做。子进程继承同一块共享页，手里的指针仍然有效。

里程碑，以及最后多出来的两件事



提交的是 Week C 的程序，A 和 B 的路径都还在。
加分：空闲读超时返回 408；`GET /api/ping` 做心跳。超时是两次 read 之间的空闲时间，默认 8 秒。

三条 curl

```
curl --http1.1 -sS -X POST http://127.0.0.1:8080/api/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"alice","password":"alice123"}'

curl --http1.1 -sS -X POST http://127.0.0.1:8080/api/messages \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"text":"..."}'

curl --http1.1 -sS http://127.0.0.1:8080/api/messages \
  -H "Authorization: Bearer $TOKEN"
```

期望依次是：200 且 body 里有 `token`；201；200 且列表里有刚写的那条。

`scripts/demo.sh` 会打印命令和结果，并检查状态码。

并发，以及不能冒充别人

- alice 和 bob 各自登录，几乎同时 `POST`。两个请求落在两个子进程里
- 之后任意一个 token 再 `GET`，列表里两条都在。这证明写进的是同一块共享内存
- body 里写了别人的用户名也没用：token 属于 alice 时，存下来的作者仍是 alice
- 不带 token 的 `POST` 得到 401，这条文字不会进列表

留言数组固定 64 条，写满后覆盖最旧的一条。演示用不了这么多。

可能会被问到的四件事

token 跨连接为什么还在？

表在共享映射上。普通全局变量确实会被 `fork` 复制。

为什么先读空行？

`send` 的边界不会保留。空行加 `Content-Length` 才是一条请求的结尾。

为什么是 401，不是 403？

401 表示还没证明你是谁。这里没有「已登录但不够权限」。

两种僵尸

子进程退出靠 `SIGCHLD`。连接迟迟不发完，靠读超时返回 408 后关掉。

现场就按这个顺序

1. `make && ./bin/restapi 8080`
2. 另一个终端：`bash scripts/demo.sh`
3. 看服务器日志：`connection -> child`，以及每条请求的 pid 和状态码

登录，发一条留言，再把列表读回来。

换端口时：`./bin/restapi 8081`，然后 `BASE=http://127.0.0.1:8081 bash scripts/demo.sh`。